Day 158 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 5 reference

Watch: Build a Spotify-Like Music Recommender System in Python (Valerio Velardo - The Sound of AI) | Mitigating the challenges of cold start in TensorFlow Recommenders (TensorFlow)

`day158_implicit_feedback.py`, segment by segment, in the order `main()` uses things. The parts (load_users, implicit_als, implicit_objective, fold_in, cooccurrence, topk_metrics) are copied unchanged into `day158_exercise.py`; Days 154 and 155's parts are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: C = confidence table (1 + α where watched, 0 elsewhere) · X, Y = user and film factors · k = factor count · lam = penalty λ · M = watched (True / False) · H = held-out films · S = scores, users × films

Segment 0 — Setup

Imports and the grids

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day154_exercise as d154
    import day155_exercise as d155
LU, LI = 5, 2
KS = (10, 20, 50, 100)
LAMS = (1, 3, 10, 30, 100, 300)
ALPHAS = (1, 3, 10, 30)
SVD_KS = (5, 10, 20, 50, 100)
REVEAL = (0, 1, 3, 5, 10, 20)
```

- `with contextlib.redirect_stdout(io.StringIO()):` — the earlier exercises print hand checks when imported; this keeps them quiet.
- `KS = (10, 20, 50, 100)` — 100 was added after the first run chose 50, the edge of the grid.
- `REVEAL = (0, 1, 3, 5, 10, 20)` — how many films are known about each brand-new user in section 5.

Segment 1 — The parts

load_users: reading u.user

CODE

```
def load_users(folder=d154.DATA):
    rows = [x.split("|") for x in open(os.path.join(folder, "ml-100k", "u.user"),
        encoding="latin-1").read().strip().split("\n")]
    occ = sorted({x[3] for x in rows})
    return (np.array([int(x[1]) for x in rows]), np.array([x[2] == "F" for x in
        rows]).astype(int),
    np.array([occ.index(x[3]) for x in rows]))
```

- `open(...).read().strip().split("\n")` — the whole file as text, outer whitespace removed, then one string per line.
- `x.split("|")` — each line's fields: id, age, gender, occupation, zip code.

- `sorted({x[3] for x in rows})` — a set of the occupations (each once), sorted, so each gets a fixed number.
- `np.array([...]).astype(int)` — True / False for “F” turned into 1 / 0.

implicit_als and implicit_objective

MATH

$$x_u = (Y^T Y + Y_w^T \, diag(c - 1) \, Y_w + \lambda I)^{-1} \, Y_w^T \, c_w$$

CODE

```
def implicit_als(C, k, lam, sweeps=10, seed=0, track=None):
    nu, ni = C.shape; rng = np.random.RandomState(seed)
    X, Y = 0.01 * rng.randn(nu, k), 0.01 * rng.randn(ni, k)
    hist = []
    for _ in range(sweeps):
        for A, B, Cm in ((X, Y, C), (Y, X, C.T)):
            G = B.T @ B + lam * np.eye(k)
            for a in range(A.shape[0]):
                j = np.flatnonzero(Cm[a]); c = Cm[a, j]
                Bj = B[j]
                A[a] = np.linalg.solve(G + (Bj * (c - 1)[:, None]).T @ Bj, Bj.T @ c)
        hist.append(track(X, Y) if track else ())
    return X, Y, hist
def implicit_objective(X, Y, C, lam):
    P = C > 0
    W = np.where(P, C, 1.0)
    return (W * (P - X @ Y.T) ** 2).sum() + lam * ((X ** 2).sum() + (Y ** 2).sum())
```

- `for A, B, Cm in ((X, Y, C), (Y, X, C.T)):` — the same code solves users (with films fixed) then films (with users fixed): C.T swaps the roles.
- `G = B.T @ B + lam * np.eye(k)` — the empty cells' part, computed once and shared by every row.
- `j = np.flatnonzero(Cm[a]); c = Cm[a, j]` — where this row has a watched film, and the confidence there.
- `(Bj * (c - 1)[:, None]).T @ Bj` — the correction for watched films: their extra weight c – 1 on top of the shared 1.
- `A[a] = np.linalg.solve(...)` — writes into X or Y in place: A is the same array, not a copy.
- `np.where(P, C, 1.0)` — the full weight table: c where watched, 1 elsewhere, used only to check the objective.

fold_in, cooccurrence and topk_metrics

MATH

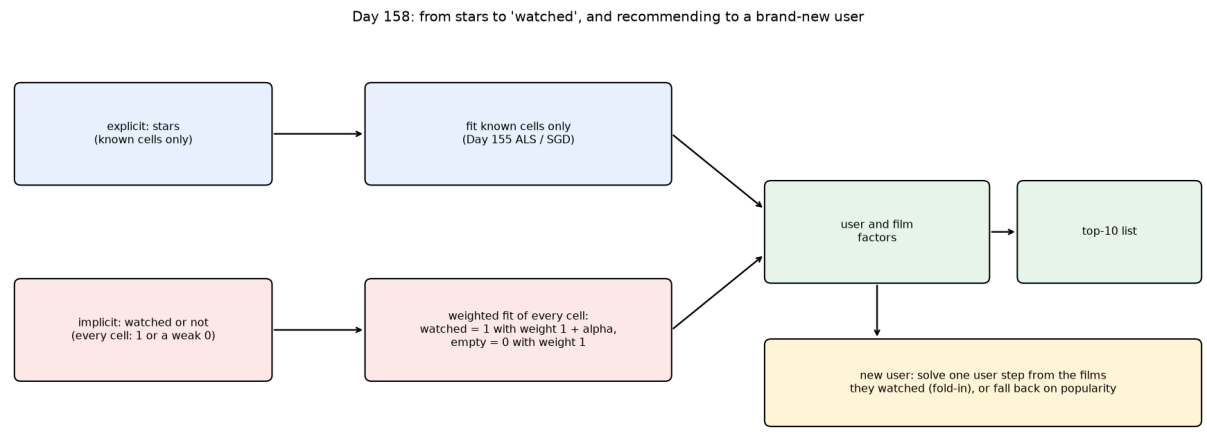
$$\cos(a, b) = (\text{watchers in common}) / \sqrt{(n_a\ n_b)};\quad \text{precision@}k = \text{hits} / k;\quad \text{recall@}k = \text{hits} / \text{held-out}$$

CODE

```
def fold_in(Y, items, conf, lam):
    k = Y.shape[1]; Bj = Y[items]
    G = Y.T @ Y + lam * np.eye(k)
    return np.linalg.solve(G + (Bj * (conf - 1)[:, None]).T @ Bj, Bj.T @ conf)
def cooccurrence(B):
    n = np.sqrt(B.sum(0))
    S = (B.T @ B) / np.maximum(np.outer(n, n), 1e-12)
    np.fill_diagonal(S, 0.0)
    return S
def topk_metrics(S, seen, held, k=10):
    S = np.where(seen, -np.inf, S); who = np.flatnonzero(held.sum(1) > 0)
    top = np.argsort(-S, axis=1, kind="stable")[:, :k]
    h = np.take_along_axis(held, top, 1)[who].sum(1)
    return h.mean() / k, (h / held[who].sum(1)).mean(), top[who]
```

- `np.maximum(np.outer(n, n), 1e-12)` — every pair's sqrt(n_a n_b), never 0, so a film nobody watched does not divide by 0.
- `np.fill_diagonal(S, 0.0)` — a film's similarity to itself set to 0, in place.
- `np.where(seen, -np.inf, S)` — already-watched films get minus infinity, so they sort last.
- `np.argsort(-S, axis=1, kind="stable")[:, :k]` — each row sorted from highest score; the first k columns are the list.
- `np.take_along_axis(held, top, 1)` — for each user, whether each listed film is held out: a hit.

SYSTEM DIAGRAM



Explicit against implicit, and fold-in for new users.

Segment 2 — The experiments

demo_ials: the grid and the comparison

CODE

```
t0 = time.time(); E = np.zeros((len(KS), len(LAMS), len(ALPHAS)))
for a1, k in enumerate(KS):
    for b1, lam in enumerate(LAMS):
        for c1, al in enumerate(ALPHAS):
    X, Y, _ = implicit_als(conf_binary(Mf, al), k, lam)
    E[a1, b1, c1] = topk_metrics(X @ Y.T, Mf, Hv)[0]
a1, b1, c1 = np.unravel_index(np.argmax(E), E.shape); k, lam, al = KS[a1], LAMS[b1], ALPHAS[c1]
Xs = X[np.random.RandomState(0).permutation(nu)]
B = M.astype(float)
scores = {"popularity": np.broadcast_to(B.sum(0), M.shape),
    "co-occurrence (item-item)": B @ cooccurrence(B),
    f"PureSVD, rank {ks}": d155.low_rank(B, ks),
    "explicit ALS (Day 155, stars)": exp,
    f"implicit ALS ({k}, {lam}, {al})": X @ Y.T,
    "control: implicit ALS, users shuffled": Xs @ Y.T}
```

- `np.zeros((len(KS), len(LAMS), len(ALPHAS)))` — a 3-D table: one cell per setting.
- `np.unravel_index(np.argmax(E), E.shape)` — argmax counts through the flattened table; this turns that count back into three indices.
- `X[np.random.RandomState(0).permutation(nu)]` — the rows of X in a random order: every user gets another user's factors.
- `np.broadcast_to(B.sum(0), M.shape)` — one row of film counts, viewed as a full table without copying it.
- `B @ cooccurrence(B)` — each user's 0/1 row times the similarity table: the sum of similarities to the films they watched.

demo_stars: stars as confidence

CODE

```
for a2 in (al / 20, al / 10, al / 5, al / 3, al / 2, al):
    X, Y, _ = implicit_als(np.where(Mf, 1 + a2 * Rf, 0.0), k, lam)
    pw, pl = topk_metrics(X @ Y.T, Mf, Hv)[0], topk_metrics(X @ Y.T, Mf, Hvl)[0]
    print(f"  alpha {a2:g} per star: {pw:.4f} / {pl:.4f}")
    if best is None or pw > best[0]:
        best = (pw, a2)
```

- `np.where(Mf, 1 + a2 * Rf, 0.0)` — confidence from the stars where watched; 0 elsewhere still means weight 1, value 0.
- `if best is None or pw > best[0]:` — keeps the best (score, setting) pair; None on the first pass.

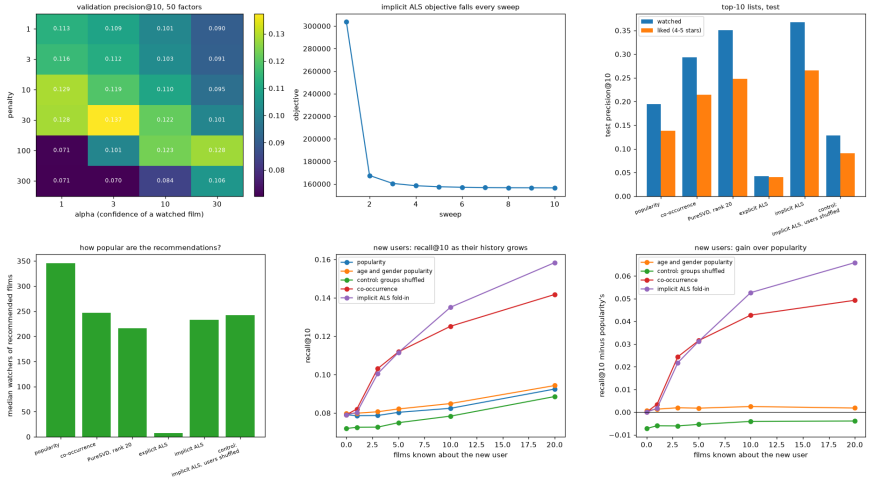
demo_cold_users: new users and what they reveal

CODE

```
band = np.digitize(age, (25, 35, 45))
grp = band * 2 + fem
gpop = np.array([B[(grp == g) & ~cold].sum(0) for g in range(8)])
sg = grp[cu][np.random.RandomState(6).permutation(len(cu))]
res = {m: [] for m in ("popularity", "age and gender popularity", "control: groups shuffled",
    "co-occurrence",
    "implicit ALS fold-in")}
for n in REVEAL:
    Rv = np.zeros((len(cu), ni), bool); rs = np.random.RandomState(5)
    for q, a in enumerate(cu):
        w = np.flatnonzero(A[a]); Rv[q, rs.choice(w, n, replace=False)] = True
    Hd = A[cu] & ~Rv
    sc = {"popularity": np.broadcast_to(cnt, Rv.shape),
    "age and gender popularity": gpop[grp[cu]] + 1e-3 * cnt,
    "control: groups shuffled": gpop[sg] + 1e-3 * cnt,
    "co-occurrence": Rv.astype(float) @ S + 1e-6 * cnt,
    "implicit ALS fold-in": np.array([fold_in(Y, np.flatnonzero(Rv[q]), np.full(n, 1.0 + al), lam)
        for q in range(len(cu))]) @ Y.T
    + 1e-6 * cnt}
    for m in res:
        res[m].append(topk_metrics(sc[m], Rv, Hd)[1])
```

- `np.digitize(age, (25, 35, 45))` — 0 for under 25, 1 for 25–34, 2 for 35–44, 3 for 45+.
- `band * 2 + fem` — 8 groups numbered 0–7: age band and gender together.
- `rs.choice(w, n, replace=False)` — n of the user's films at random, none twice.
- `A[cu] & ~Rv` — what the new users watched, minus what was revealed: the films to find.
- `gpop[grp[cu]]` — fancy indexing: each new user's row is their group's film counts.
- `+ 1e-6 * cnt` — a tiny nudge so that ties (every score 0 when nothing is known) are broken by popularity.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.flatnonzero(a)` — positions of the non-zero entries.
- `np.unravel_index(i, shape)` — a flat position back to indices.
- `np.broadcast_to(a, shape)` — a read-only full-size view.

- `np.take_along_axis(a, idx, 1)` — pick per row by index.
- `np.fill_diagonal(S, 0)` — set the diagonal in place.
- `np.digitize(x, edges)` — which band each value falls in.
- `rng.choice(w, n, replace=False)` — n distinct picks.
- `{x for x in rows}` — a set comprehension.

End of Day 158 syntax reference. For the experiments and what they showed, see the Day 158 complete guide.